

PRÁTICA 01 · SEMINÁRIO

unittest vs pytest

Comparando duas bibliotecas de testes unitários em Python: pontos fortes, pontos fracos e diferenças na prática.

Linguagem: Python • Bibliotecas: unittest (nativa) e pytest (externa)

O que são testes unitários?

1

Verificam uma unidade isolada de código

Uma função, método ou classe, testada separadamente do restante do sistema.

2

Automatizam a verificação

Rodam em segundos e podem ser repetidos a cada alteração no código.

3

Detectam erros cedo

Problemas são encontrados antes de chegar à produção, reduzindo custo de correção.

4

Servem de documentação viva

Mostram como cada parte do código deve se comportar.

Em Python, as bibliotecas mais usadas para isso são o unittest (nativo) e o pytest (externo).

unittest — visão geral

Módulo nativo da biblioteca padrão do Python, inspirado no JUnit (Java). Organiza os testes em classes que herdam de `unittest.TestCase`.

PONTOS POSITIVOS

- Já vem instalado — não precisa de pip install
- Estrutura orientada a objetos (`setUp/tearDown`)
- Amplamente usado em projetos legados/corporativos
- Muitos métodos assert prontos (`assertEqual`, `assertTrue`...)

PONTOS NEGATIVOS

- Sintaxe mais verbosa e burocrática
- Exige criar classes mesmo para testes simples
- Mensagens de erro menos detalhadas
- Fixtures (`setUp/tearDown`) menos flexíveis que as do `pytest`

pytest — visão geral

Framework externo (pip install pytest), muito popular na comunidade Python. Usa funções simples com o assert nativo da linguagem, sem precisar de classes.

PONTOS POSITIVOS

- Sintaxe simples: usa assert puro do Python
- Não exige classes — funções soltas já funcionam
- Mensagens de erro muito mais detalhadas
- Fixtures poderosas e reutilizáveis
- Também roda testes escritos em unittest

PONTOS NEGATIVOS

- Precisa ser instalado separadamente (pip install)
- Não faz parte da biblioteca padrão do Python
- Muitos plugins podem gerar dependências extras
- Curva de aprendizado leve para usar fixtures avançadas

Exemplo de código — mesma função testada

Função a ser testada: `soma(a, b)`, que retorna $a + b$.

test_calculadora_unittest.py X

test_calculadora_unittest.py > ...

```
1  import unittest
2  from calculadora import soma
3
4
5  class TestSoma(unittest.TestCase):
6
7      def test_soma_positivos(self):
8          self.assertEqual(soma(2, 3), 6)
9
10     def test_soma_negativos(self):
11         self.assertEqual(soma(-1, -1), -2)
12
13
14  if __name__ == '__main__':
15     unittest.main()
16
```

test_calculadora_pytest.py X

test_calculadora_pytest.py > ...

```
1  from calculadora import soma
2
3
4  def test_soma_positivos():
5      assert soma(2, 3) == 6
6
7
8  def test_soma_negativos():
9      assert soma(-1, -1) == -2
10
```


Diferenças na saída — execução com sucesso

terminal – unittest

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(.venv) PS C:\Users\jotin\Desktop\Seminario 04.09.24> python -m unittest test_calculadora_unittest.py -v
test_soma_negativos (test_calculadora_unittest.TestSoma.test_soma_negativos) ... ok
test_soma_positivos (test_calculadora_unittest.TestSoma.test_soma_positivos) ... ok

-----
Ran 2 tests in 0.001s
```

Diferenças na saída — execução com sucesso

terminal – pytest

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(.venv) PS C:\Users\jotin\Desktop\Seminario 04.09.24> pytest -v test_calculadora_pytest.py
===== test session starts =====
platform win32 -- Python 3.12.6, pytest-9.1.1, pluggy-1.6.0 -- c:\Users\jotin\Desktop\Seminario 04.09.24\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\jotin\Desktop\Seminario 04.09.24
collected 2 items

test_calculadora_pytest.py::test_soma_positivos PASSED [ 50%]
test_calculadora_pytest.py::test_soma_negativos PASSED [100%]

===== 2 passed in 0.03s =====
```

Diferenças na saída — quando um teste falha

Alterando o teste para esperar soma(2, 3) == 6 (valor errado, propositalmente):

terminal – unittest

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(.venv) PS C:\Users\jotin\Desktop\Seminario 04.09.24> python -m unittest test_calculadora_unittest.py -v
test_soma_negativos (test_calculadora_unittest.TestSoma.test_soma_negativos) ... ok
test_soma_positivos (test_calculadora_unittest.TestSoma.test_soma_positivos) ... FAIL

=====
FAIL: test_soma_positivos (test_calculadora_unittest.TestSoma.test_soma_positivos)
-----
Traceback (most recent call last):
  File "C:\Users\jotin\Desktop\Seminario 04.09.24\test_calculadora_unittest.py", line 8, in test_soma_positivos
    self.assertEqual(soma(2, 3), 6)
AssertionError: 5 != 6

-----

Ran 2 tests in 0.001s

FAILED (failures=1)
(.venv) PS C:\Users\jotin\Desktop\Seminario 04.09.24> █
```


Diferenças na saída — quando um teste falha

Alterando o teste para esperar soma(2, 3) == 6 (valor errado, propositalmente):

terminal – pytest

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(.venv) PS C:\Users\jotin\Desktop\Seminario 04.09.24> pytest -v test_calculadora_pytest.py
===== test session starts =====
platform win32 -- Python 3.12.6, pytest-9.1.1, pluggy-1.6.0 -- c:\Users\jotin\Desktop\Seminario 04.09.24\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\jotin\Desktop\Seminario 04.09.24
collected 2 items

test_calculadora_pytest.py::test_soma_positivos FAILED [ 50%]
test_calculadora_pytest.py::test_soma_negativos PASSED [100%]

===== FAILURES =====
_____ test_soma_positivos _____

    def test_soma_positivos():
>     assert soma(2, 3) == 6
E       assert 5 == 6
E       + where 5 = soma(2, 3)

test_calculadora_pytest.py:5: AssertionError

===== short test summary info =====
FAILED test_calculadora_pytest.py::test_soma_positivos - assert 5 == 6
===== 1 failed, 1 passed in 0.26s =====

(.venv) PS C:\Users\jotin\Desktop\Seminario 04.09.24> 
```

Tabela comparativa

Critério	unittest	pytest
Instalação	Nativo (nenhuma)	pip install pytest
Sintaxe	Classes + assertEquals...	Funções + assert nativo
Boilerplate	Alto (herda TestCase)	Baixo (função simples)
Mensagens de erro	Básicas	Detalhadas (introspecção)
Fixtures	setUp / tearDown	@pytest.fixture (flexível)
Compatibilidade	Padrão da linguagem	Executa testes unittest também
Curva de aprendizado	Moderada	Baixa para o básico

Quando usar cada uma?

unittest

- Projetos que não podem ter dependências externas
- Equipes/empresas que já usam esse padrão
- Quando se quer algo 100% nativo do Python

pytest

- Projetos novos, times ágeis e produtividade no dia a dia
- Quando facilidade de leitura e depuração importam
- Testes mais complexos, com fixtures e parametrização

Na prática, o pytest é hoje a escolha mais comum em projetos novos por sua simplicidade mas o unittest continua relevante por já vir pronto no Python.